



Instituto Superior de Engenharia

Politécnico de Coimbra

DEPARTAMENTO DE / DEPARTMENT OF
INFORMÁTICA E SISTEMAS

Trabalho Prático Nº2 - Problema de Otimização Introdução à Inteligência Artificial

Autor / Author

David Martins Correia – 2023140410

Tiago Gouveia Filipe – 2019112767



INSTITUTO POLITÉCNICO DE
COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Coimbra, 20 outubro de 2024

ÍNDICE

Introdução.....	4
Métodos de estudo Aplicados	5
Trepa Colinas.....	5
Evolutivo.....	5
Híbrido	5
Funções implementadas.....	6
Método Aplicado Penalização.....	6
Método Aplicado Reparação.....	6
Vizinho 1	6
Vizinho 2.....	6
Crossover	7
Crossover2	7
Mutação 1.....	7
Mutação 2.....	7
Estudos dos resultados.....	8
Pesquisa local.....	8
Vizinho 1	8
Vizinho 2.....	10
Método Evolutivo	12
Torneio 1.....	12
Torneio 2.....	14
Híbrido	16
Híbrido 1 / 2	16
Conclusão	17

Introdução

Neste trabalho prático, desenvolvido na unidade curricular de Introdução à Inteligência Artificial, o objetivo é implementar e avaliar algoritmos de otimização para resolver o Problema da Troca de Moedas. Este problema consiste em encontrar a combinação de moedas que, juntas, somem a um valor específico com o menor número de moedas possível.

Os métodos de otimização utilizados incluem algoritmos de pesquisa local, algoritmos evolutivos e combinações híbridas de ambos. Através de testes com diferentes instâncias do problema, analisamos a eficiência e a eficácia de cada abordagem na procura de soluções de alta qualidade.

Métodos de estudo Aplicados

Trepa Colinas

Este método utiliza uma abordagem de procura local para encontrar a melhor solução. Começando com uma solução inicial criada aleatoriamente e, em seguida, explora soluções vizinhas e tipos de penalidade ou reparação para encontrar uma com um custo menor. O custo médio das soluções válidas e a melhor solução encontrada são exibidos no final.

Evolutivo

Este método implementa um algoritmo genético. Ele começa por criar uma população inicial de soluções e, em seguida, avalia a qualidade de cada solução. A partir daí, utiliza um torneio binário ou global para selecionar soluções para a próxima geração. Operadores genéticos, como crossover e mutação, são aplicados para gerar soluções. O processo é repetido por um número definido de gerações, e no final, o custo médio e a melhor solução encontrada são apresentados.

Híbrido

Este método combina elementos do algoritmo Trepa-Colinas e do algoritmo evolutivo. Inicialmente, ele gera uma população de soluções e aplica o algoritmo Trepa-Colinas (apenas no híbrido 2) a cada solução para melhorar sua qualidade. Em seguida, utiliza o algoritmo evolutivo para criar gerações de soluções. Após várias gerações, o algoritmo Trepa-Colinas é aplicado novamente à melhor solução encontrada para tentar encontrar a melhor solução otimizada. O custo médio e a melhor solução são exibidos ao final.

Funções implementadas

Método Aplicado Penalização

A penalidade na implementação é aplicada para avaliar a qualidade de uma solução em relação ao valor a atingir. Quando a solução não atinge o valor desejado (ou seja, a soma dos custos das moedas não é igual ao valor a atingir), a qualidade da solução é penalizada, atribuindo um valor alto (como INT_MAX) à qualidade. Isto é feito na função `calcula_fit`, onde se verifica a diferença entre o custo total e o valor a atingir. Se a diferença for maior que um pequeno limiar (0.0001), a solução é considerada inválida e recebe uma penalização severa, não sendo aceite para a realização dos processos de seleção ou de otimização.

Apesar destes valores não serem utilizados para o cálculo de media nem de melhor valor atingido, podem influenciar na solução, o que por sua vez pode provocar casos de estudo inválidos.

Método Aplicado Reparação

A reparação na implementação é um processo que visa corrigir soluções inválidas, ou seja, aquelas que não atingem o valor desejado. Na função `calcula_fit2`, se a soma dos custos das moedas não corresponder ao valor a atingir, a função ajusta a solução. Se o custo total for menor que o valor desejado, a função adiciona moedas aleatoriamente até que o custo atinja o valor correto. Se o custo total for maior, a função remove moedas aleatoriamente. Este processo continua até que a solução se torne válida, permitindo que soluções inicialmente inadequadas sejam transformadas em soluções viáveis.

Vizinho 1

O "vizinho 1" é gerado pela função `create_neighbor`, que cria uma solução a partir da solução atual. Primeiro, a função copia a solução atual para uma nova solução. Em seguida, escolhe aleatoriamente uma ação: adicionar ou remover uma moeda, um índice de moeda também é selecionado aleatoriamente. Se a ação for adicionar, a função incrementa o número de moedas neste índice; se for remover, ela decrementa, desde que existam moedas disponíveis. O resultado é uma nova solução que representa uma pequena variação da solução atual.

Vizinho 2

O "vizinho 2" é gerado pela função `create_neighbor2`, que cria uma solução a partir da solução atual, mas com uma abordagem diferente. A função começa por copiar a solução atual para uma nova solução. Em seguida, executa um loop que itera duas vezes, onde em cada interação escolhe aleatoriamente uma ação (adicionar ou remover uma moeda) e um índice de moeda. Se a ação for adicionar, a função incrementa o número de moedas neste

índice, se for remover, decrementa, desde que existam moedas disponíveis. O resultado é uma nova solução que pode ter até duas moedas adicionadas ou removidas, permitindo uma exploração mais ampla do espaço de soluções durante o processo de otimização.

Crossover

Crossover de Um Ponto: A função crossover escolhe um ponto de corte aleatório em duas soluções. Se a probabilidade de recombinação (definida por `prob_recomb`) for atendida, as partes das soluções são trocadas a partir deste ponto, com a criação de duas novas soluções. Se a probabilidade não for atendida, as soluções são copiadas diretamente para os descendentes.

Crossover2

A função crossover2 funciona de maneira semelhante, mas utiliza dois pontos de corte. Isto permite uma troca mais complexa entre as soluções. As partes das soluções entre os dois pontos de corte são trocadas, enquanto as partes antes do primeiro ponto e após o segundo ponto permanecem inalteradas. Se a probabilidade de recombinação não for atendida, as soluções são copiadas diretamente.

Mutação 1

A função mutation aplica uma mutação a cada solução com base em uma probabilidade definida (`prob_mut`). Se a condição for atendida, uma nova solução é gerada usando a função `create_neighbor`, que altera a solução atual ao adicionar ou remover uma moeda. A nova solução substitui a antiga. Esta abordagem permite pequenas alterações, que ajuda a evitar que o algoritmo fique preso em soluções locais.

Mutação 2

A função mutation2 segue um processo semelhante, mas utiliza a função `create_neighbor2`, que pode adicionar ou remover até duas moedas. Isto resulta em uma variação maior nas soluções, permitindo uma exploração mais ampla do espaço de soluções. Assim como na mutação simples, a nova solução gerada substitui a antiga se a condição de probabilidade for atendida.

Estudos dos resultados

Pesquisa local

Vizinho 1

Evolutivo com mutação 1 cross-over 1 e torneio 1									
		file1.txt		file1.txt Com Penalização		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100	pr = 0.3	2	2.90000	2	544.00000	20	23.76667	4	5.30000
pm = 0.01	pr = 0.5 pr = 0.7	2	2.50000		497.00000	20	24.96667	4	5.00000
		2	2.70000	2	601.00000	20	23.46667	4	4.90000
pop = 100	pm = 0.0	2	2.00000	2	456.00000	20	21.53333	4	4.00000
pr = 0.7	pm = 0.01 pm = 0.05	2	2.60000	2	384.00000	20	27.63333	4	5.16667
		2	5.40000	2	444.00000	20	26.90000	4	8.06667
pr = 0.7 pm = melhor valor obtido	pop = 5	2	2.40000	2	442.00000	20	32.30000	4	4.03333
	pop = 50	2	2.00000	2	669.00000	20	22.33333	4	4.00000
	pop = 100	2	2.00000	2	530.00000	20	21.53333	4	4.00000

Evolutivo com mutação 1 cross-over 2 e torneio 1

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100 pm = 0.01	pr = 0.3	2	2.70000	20	26.40000	4	5.60000
	pr = 0.5	2	3.00000	20	21.96667	4	5.50000
	pr = 0.7	2	2.60000	20	21.86667	4	4.90000
pop = 100 pr = 0.7	pm = 0.0	2	2.00000	20	20.40000	4	4.00000
	pm = 0.01	2	3.50000	20	22.13333	4	4.66667
	pm = 0.05	2	4.20000	20	25.93333	4	8.73333
pr = 0.7 pm = melhor valor obtido	pop = 5	2	2.20000	20	30.53333	4	4.33333
	pop = 50	2	2.00000	20	21.83333	4	4.00000
	pop = 100	2	2.00000	20	20.43333	4	4.00000

Evolutivo com mutação 2 cross-over 1 e torneio 1

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100 pm = 0.01	pr = 0.3	2	3.70000	20	25.86667	4	5.20000
	pr = 0.5	2	3.00000	20	24.36667	4	4.63333
	pr = 0.7	2	3.10000	20	23.23333	4	4.86667
pop = 100 pr = 0.7	pm = 0.0	2	2.00000	20	23.03333	4	4.00000
	pm = 0.01	2	3.60000	20	24.56667	4	5.00000
	pm = 0.05	2	5.30000	20	28.36667	4	8.10000
pr = 0.7 pm = melhor valor obtido	pop = 5	2	2.40000	20	31.80000	4	4.30000
	pop = 50	2	2.00000	20	22.20000	4	4.00000
	pop = 100	2	2.00000	20	21.66667	4	4.00000

Evolutivo com mutação 2 cross-over 2 e torneio 1

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100 pm = 0.01	pr = 0.3	2	3.90000	20	24.46667	4	6.06667
	pr = 0.5	2	2.70000	20	22.90000	4	5.33333
	pr = 0.7	2	3.00000	20	22.96667	4	5.30000
pop = 100 pr = 0.7	pm = 0.0	2	2.00000	20	21.26667	4	4.00000
	pm = 0.01	2	3.70000	20	23.43333	4	5.23333
	pm = 0.05	2	5.00000	20	26.60000	4	8.26667
pr = 0.7 pm = melhor valor obtido	pop = 5	2	2.50000	20	30.76667	4	4.16667
	pop = 50	2	2.00000	20	22.43333	4	4.00000
	pop = 100	2	2.00000	20	21.20000	4	4.00000

Com base nos dados apresentados, podemos concluir que a estratégia de recompensação tende a produzir melhores resultados em comparação com a penalização. A aceitação de vizinhos iguais não parece afetar significativamente os resultados quando a recompensação é utilizada, mantendo a consistência nos valores. O aumento no número de interações geralmente melhora a qualidade das soluções, especialmente visível nos arquivos file2.txt e file3.txt. No entanto, para alguns arquivos, como file1.txt e file5.txt, o aumento das interações não resulta em melhorias significativas, sugerindo que o algoritmo pode estar convergindo rapidamente para uma solução ótima ou próxima do ótimo. A aplicação do método da penalidade neste caso não é relevante uma vez que os valores se distanciam dos valores ótimos quando comparados com a reparação, nos restantes estudos já não efetuamos esta pesquisa. Em resumo, a estratégia de recompensação com aceitação de vizinhos iguais parece ser a mais eficaz, especialmente para instâncias mais complexas. Aumentar o número de interações pode melhorar a qualidade das soluções, mas o impacto varia dependendo da complexidade do problema.

Vizinho 2

Trepa-Colinas com Vizinhança 2 penalização não aceita vizinhos iguais

NUM VERT		100 it	1000 it	5000 it	10000 it
file1.txt	Melhor	2	2	2	2
	MBF	4.17241	3.60714	4.33333	4.78571
file2.txt	Melhor	31	30	27	30
	MBF	51.11765	45.00000	46.26667	47.93750
file3.txt	Melhor	161	163	173	163
	MBF	191.14285	183.81250	184.33333	185.70833
file4.txt	Melhor	319	247	323	319
	MBF	399.37500	373.88889	402.15790	420.38889
file5.txt	Melhor	5	5	5	4
	MBF	8.42857	8.66666	8.60000	6.46154

Trepa-Colinas com Vizinhança 2 recompensação não aceita vizinhos iguais

NUM VERT		100 it	1000 it	5000 it	10000 it
file1.txt	Melhor	2	2	2	2
	MBF	2.00000	2.00000	2.00000	2.00000
file2.txt	Melhor	20	20	20	20
	MBF	20.20000	20.00000	20.00000	20.00000
file3.txt	Melhor	102	102	102	102
	MBF	119.56667	102.00000	102.00000	102.00000
file4.txt	Melhor	109	108	108	108
	MBF	158.93333	108.00000	108.00000	108.00000
file5.txt	Melhor	4	4	4	4
	MBF	4.00000	4.00000	4.00000	4.00000

Trepa-Colinas com Vizinhança 2 penalização aceita vizinhos iguais

NUM VERT		100 it	1000 it	5000 it	10000 it
file1.txt	Melhor	2	2	2	2
	MBF	3.62500	5.82759	5.20690	4.16000
file2.txt	Melhor	28	29	36	35
	MBF	42.57143	49.86364	59.53846	60.16000
file3.txt	Melhor	177	167	115	153
	MBF	189.66667	189.52000	183.64000	184.96552
file4.txt	Melhor	315	247	268	310
	MBF	357.50000	365.33334	380.10001	407.71429
file5.txt	Melhor	4	5	7	5
	MBF	8.00000	5.00000	9.20000	8.50000

Trepa-Colinas com Vizinhança 2 recompensação aceita vizinhos iguais

NUM VERT		100 it	1000 it	5000 it	10000 it
file1.txt	Melhor	2	2	2	2
	MBF	2.00000	2.00000	2.00000	2.00000
file2.txt	Melhor	20	20	20	20
	MBF	20.20000	20.00000	20.00000	20.00000
file3.txt	Melhor	102	102	102	102
	MBF	121.46667	102.00000	102.00000	102.00000
file4.txt	Melhor	111	108	108	108
	MBF	157.36667	108.00000	108.00000	108.00000
file5.txt	Melhor	4	4	4	4
	MBF	4.00000	4.00000	4.00000	4.00000

Podemos concluir que, para o método de Trepa-Colinas com Vizinhança 2, as observações são similares às do método de Vizinhança 1. A estratégia de recompensação com aceitação de vizinhos iguais demonstrou ser mais eficaz em ambos os casos, especialmente em instâncias mais complexas. Além disso, o aumento do número de interações tende a melhorar a qualidade das soluções, embora o impacto varie conforme a complexidade do problema. Portanto, as conclusões anteriormente alcançadas para o método de Vizinhança 1 também se aplicam ao método de Vizinhança 2.

Método Evolutivo

Torneio 1

Evolutivo com mutação 1 cross-over 1 e torneio 1

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100	pr = 0.3	2	2.90000	20	23.76667	4	5.30000
pm = 0.01	pr = 0.5	2	2.50000	20	24.96667	4	5.00000
	pr = 0.7	2	2.70000	20	23.46667	4	4.90000
pop = 100	pm = 0.0	2	2.00000	20	21.53333	4	4.00000
pr = 0.7	pm = 0.01	2	2.60000	20	27.63333	4	5.16667
	pm = 0.05	2	5.40000	20	26.90000	4	8.06667
pr = 0.7	pop = 5	2	2.40000	20	32.30000	4	4.03333
pm = melhor valor obtido	pop = 50	2	2.00000	20	22.33333	4	4.00000
	pop = 100	2	2.00000	20	21.53333	4	4.00000

Evolutivo com mutação 1 cross-over 2 e torneio 1

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100	pr = 0.3	2	2.70000	20	26.40000	4	5.60000
pm = 0.01	pr = 0.5	2	3.00000	20	21.96667	4	5.50000
	pr = 0.7	2	2.60000	20	21.86667	4	4.90000
pop = 100	pm = 0.0	2	2.00000	20	20.40000	4	4.00000
pr = 0.7	pm = 0.01	2	3.50000	20	22.13333	4	4.66667
	pm = 0.05	2	4.20000	20	25.93333	4	8.73333
pr = 0.7	pop = 5	2	2.20000	20	30.53333	4	4.33333
pm = melhor valor obtido	pop = 50	2	2.00000	20	21.83333	4	4.00000
	pop = 100	2	2.00000	20	20.43333	4	4.00000

Evolutivo com mutação 2 cross-over 1 e torneio 1

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100	pr = 0.3	2	3.70000	20	25.86667	4	5.20000
pm = 0.01	pr = 0.5	2	3.00000	20	24.36667	4	4.63333
	pr = 0.7	2	3.10000	20	23.23333	4	4.86667
pop = 100	pm = 0.0	2	2.00000	20	23.03333	4	4.00000
pr = 0.7	pm = 0.01	2	3.60000	20	24.56667	4	5.00000
	pm = 0.05	2	5.30000	20	28.36667	4	8.10000
pr = 0.7	pop = 5	2	2.40000	20	31.80000	4	4.30000
pm = melhor valor obtido	pop = 50	2	2.00000	20	22.20000	4	4.00000
	pop = 100	2	2.00000	20	21.66667	4	4.00000

Evolutivo com mutação 2 cross-over 2 e torneio 1

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100	pr = 0.3	2	3.90000	20	24.46667	4	6.06667
pm = 0.01	pr = 0.5	2	2.70000	20	22.90000	4	5.33333
	pr = 0.7	2	3.00000	20	22.96667	4	5.30000
pop = 100	pm = 0.0	2	2.00000	20	21.26667	4	4.00000
pr = 0.7	pm = 0.01	2	3.70000	20	23.43333	4	5.23333
	pm = 0.05	2	5.00000	20	26.60000	4	8.26667
pr = 0.7	pop = 5	2	2.50000	20	30.76667	4	4.16667
pm = melhor valor obtido	pop = 50	2	2.00000	20	22.43333	4	4.00000
	pop = 100	2	2.00000	20	21.20000	4	4.00000

Podemos concluir que variar a taxa de cruzamento (p_r) entre 0.3, 0.5 e 0.7, que normalmente a melhor opção é 0.7, uma vez que este tende a ter a média de resultados mais próxima do ótimo.

Quando a taxa de mutação (p_m) é 0.0, os resultados tendem a ser mais consistentes e próximos do ótimo. Aumentar a taxa de mutação geralmente leva a uma maior variação nos resultados, mais distantes do ótimo.

Variar o tamanho da população entre 5, 50 e 100 mostra que populações maiores tendem a proporcionar resultados melhores e mais consistentes.

A combinação de operadores de mutação e cross-over influencia os resultados, mas as diferenças não são drásticas. Os métodos mutação 1 e cross-over 1 geralmente apresentam resultados ligeiramente melhores em relação aos valores da média em comparação com as outras combinações.

Em resumo, a otimização do método evolutivo mostra que a escolha de uma taxa de cruzamento moderada (0.7) e de uma taxa de mutação baixa (0.0), juntamente com uma população maior, tem tendência em produzir os melhores resultados. É importante notar que, como p_r e p_m são probabilidades, há sempre uma chance de variação nos resultados. A combinação dos operadores pode ser ajustada para melhorar ligeiramente os resultados, apesar da diferença não ser significativa.

Torneio 2

Evolutivo com mutação 1 cross-over 1 e torneio 2

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100	pr = 0.3	2	2.80000	20	24.13333	4	5.10000
pm = 0.01	pr = 0.5	2	2.90000	20	24.56667	4	4.83333
	pr = 0.7	2	2.50000	20	22.10000	4	4.70000
pop = 100	pm = 0.0	2	2.00000	20	21.06667	4	4.00000
pr = 0.7	pm = 0.01	2	2.60000	20	23.03333	4	5.36667
	pm = 0.05	2	4.60000	20	26.13333	4	7.66667
pr = 0.7	pop = 5	2	2.30000	20	32.33333	4	4.33333
pm = melhor valor obtido	pop = 50	2	2.00000	20	22.96667	4	4.00000
	pop = 100	2	2.00000	20	21.96667	4	4.00000

Evolutivo com mutação 1 cross-over 2 e torneio 2

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100	pr = 0.3	2	2.70000	20	25.56667	4	5.60000
pm = 0.01	pr = 0.5	2	3.10000	20	22.56667	4	4.96667
	pr = 0.7	2	2.60000	20	22.33333	4	5.23333
pop = 100	pm = 0.0	2	2.00000	20	22.53333	4	4.00000
pr = 0.7	pm = 0.01	2	2.80000	20	21.93333	4	5.20000
	pm = 0.05	2	5.50000	20	27.66667	4	7.76667
pr = 0.7	pop = 5	2	2.30000	20	31.96667	4	4.30000
pm = melhor valor obtido	pop = 50	2	2.00000	20	21.46667	4	4.00000
	pop = 100	2	2.00000	20	20.63333	4	4.00000

Evolutivo com mutação 2 cross-over 1 e torneio 2

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100	pr = 0.3	2	3.60000	20	24.56667	4	5.10000
pm = 0.01	pr = 0.5	2	3.60000	20	24.23333	4	4.76667
	pr = 0.7	2	2.80000	20	23.53333	4	5.66667
pop = 100	pm = 0.0	2	2.00000	20	22.06667	4	4.00000
pr = 0.7	pm = 0.01	2	3.20000	20	24.00000	4	4.00000
	pm = 0.05	2	4.80000	20	28.06667	4	5.40000
pr = 0.7	pop = 5	2	2.40000	20	31.90000	4	4.36667
pm = melhor valor obtido	pop = 50	2	2.00000	20	22.63333	4	4.00000
	pop = 100	2	2.00000	20	22.93333	4	4.00000

Evolutivo com mutação 2 cross-over 2 e torneio 2

		file1.txt		file2.txt		file5.txt	
Parâmetros Fixos	Parâmetros a variar	Melhor	MBF	Melhor	MBF	Melhor	MBF
pop = 100	pr = 0.3	2	3.20000	20	25.43333	4	5.73333
pm = 0.01	pr = 0.5	2	2.90000	20	24.90000	4	6.03333
	pr = 0.7	2	2.50000	20	24.80000	4	5.50000
pop = 100	pm = 0.0	2	2.00000	20	21.26667	4	4.00000
pr = 0.7	pm = 0.01	2	2.60000	20	22.20000	4	5.20000
	pm = 0.05	2	6.50000	20	29.80000	4	8.50000
pr = 0.7	pop = 5	2	2.20000	20	30.70000	4	4.26667
pm = melhor valor obtido	pop = 50	2	2.00000	20	22.63333	4	4.00000
	pop = 100	2	2.00000	20	22.20000	4	4.00000

Em resumo, as conclusões para este conjunto de dados são semelhantes às do anterior. Variar a taxa de cruzamento e a taxa de mutação, bem como o tamanho da população, apresenta resultados consistentes. Portanto, a escolha de uma taxa de cruzamento moderada (0.7), uma taxa de mutação baixa (0.0) e uma população maior tende a produzir os melhores resultados, tal como observado anteriormente.

Híbrido

Híbrido 1 / 2

Híbrido				
File1.txt	Algoritmo base híbrido i)		Algoritmo base híbrido ii)	
Parâmetros Fixos	Best	MBF	Best	MBF
pop = 100	2,0	2.00000	2,0	2.00000
pr = 0.7				
pm = 0.001				
File2.txt	Algoritmo base híbrido i)		Algoritmo base híbrido ii)	
Parâmetros Fixos	Best	MBF	Best	MBF
pop = 100	20,0	20.00000	20,0	20.00000
pr = 0.7				
pm = 0.001				
File3.txt	Algoritmo base híbrido i)		Algoritmo base híbrido ii)	
Parâmetros Fixos	Best	MBF	Best	MBF
pop = 100	102,0	102.00000	102,0	102.00000
pr = 0.7				
pm = 0.001				
File4.txt	Algoritmo base híbrido i)		Algoritmo base híbrido ii)	
Parâmetros Fixos	Best	MBF	Best	MBF
pop = 100	108,0	108.00000	108,0	108.00000
pr = 0.7				
pm = 0.001				
File5.txt	Algoritmo base híbrido i)		Algoritmo base híbrido ii)	
Parâmetros Fixos	Best	MBF	Best	MBF
pop = 100	4,0	4.00000	4,0	4.00000
pr = 0.7				
pm = 0.001				

A partir dos resultados do método híbrido, podemos observar que ambos os algoritmos híbridos (i) e (ii) apresentam um desempenho consistente e ótimo. Em todos os arquivos de teste (file1.txt, file2.txt, file3.txt, file4.txt e file5.txt), os melhores valores e o valor da media, são idênticos entre os dois algoritmos, indicando que ambos são igualmente eficazes.

Em resumo, os métodos híbridos implementados mostram-se altamente eficazes e consistentes, alcançando soluções ótimas com os parâmetros definidos.

Conclusão

Em suma, o trepa-colinas destacou-se pela sua capacidade de encontrar soluções rapidamente, especialmente quando utilizado com a estratégia de recompensação e aceitação de vizinhos iguais. Esta abordagem mostrou-se eficaz em instâncias mais complexas, melhorando a qualidade das soluções ao longo das interações.

Os algoritmos evolutivos, por sua vez, demonstraram uma melhor robustez, quando feito com reparação, permitindo a exploração de um espaço de soluções mais amplo através de operadores como o crossover e a mutação. Embora tenham produzido resultados consistentes, a sua eficácia depende da combinação de operadores utilizados.

Por fim, os métodos híbridos combinam as vantagens das duas abordagens anteriores, resultando em um desempenho otimizado e consistente em todas as instâncias testadas. Ambos os algoritmos híbridos alcançaram soluções idênticas, indicando que a integração de técnicas pode ser uma estratégia preponderante para resolver problemas complexos.

Em resumo, enquanto o Tropa-Colinas é eficaz em encontrar soluções rapidamente, os algoritmos evolutivos oferecem uma exploração mais ampla, e os métodos híbridos combinam o melhor de ambos, mostrando-se como uma abordagem mais robusta e eficaz para o Problema da Troca de Moedas.